

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ГЕОДЕЗИИ И
КАРТОГРАФИИ»
(ММИГАиК)
Факультет геоинформатики и информационной безопасности
(ФГиИБ)

ПРАКТИЧЕСКАЯ РАБОТА №2

по дисциплине: Системы сбора больших пространственных данных
на тему: «Большие пространственные данные в сфере строительства»

Выполнил студент
Первого курса магистратуры
Группы 2025-ФГиИБ-ПИ-1м
Сабитов Р.Р.

Москва, 2026 г.

Содержание

1. Введение.....	3
2. Архитектура разработанной системы	4
3. Сбор больших пространственных данных из OSM.....	5
4. Пространственно-временной анализ: KDE и аномалии.....	7
4.1. Метод Kernel Density Estimation	7
4.2. Годовая статистика и кластеризация.....	7
5. Визуализация и финальные результаты.....	9
5.1. Статические карты KDE и динамика	9
5.2. Интерактивная карта в Folium	9
5.3. CSV, Geopackage и текстовый отчет	10
6. Заключение.....	12
Список использованных источников	13

1. Введение

Цель работы — разработка программной системы, которая автоматически собирает данные о строительной активности из OpenStreetMap (OSM), выполняет пространственно-временной анализ (Kernel Density Estimation, KDE) и выявляет зоны аномальной строительной активности в городах. Такая система позволяет опираться на большие открытые геоданные, а не на ручные проверки и разрозненные ведомственные источники, что особенно актуально для мониторинга застройки и контроля потенциально незаконных объектов.

В качестве исходных данных используются контуры зданий OSM, дополнительные атрибуты (в первую очередь `start_date` как прокси для времени появления объекта), а также геометрические характеристики (площадь, координаты центроида). Программная реализация выполнена на Python с использованием библиотек `osmnx`, `GeoPandas`, `Shapely`, `SciPy`, `scikit-learn`, `matplotlib` и `folium`.

2. Архитектура разработанной системы

Система реализована как модульный

Python-проект `gis_construction_system` со следующей структурой:

```
gis_construction_system/  
├── config.py          # Параметры анализа (города, годы, пороги KDE)  
├── collector.py       # Сбор и предобработка геоданных из OSM  
├── analyzer.py        # Пространственно-временной анализ и поиск аномалий  
├── visualizer.py      # Статические и интерактивные карты  
├── reporter.py        # CSV/GeoPackage/текстовые отчёты  
├── main.py            # CLI-оболочка и оркестрация пайплайна  
├── data/              # Кэш OSM-данных (GeoPackage)  
├── output/            # PNG/HTML/CSV результаты  
└── reports/           # Текстовые отчёты и статистика
```

Конфигурационный модуль `config.py` задаёт перечень городов, годы анализа, параметры KDE (`bandwidth` и порог `z-score`), системы координат и пути к каталогам. Это позволяет запускать один и тот же код для разных городов, меняя только значения в словаре `CITIES` и диапазон лет. Точка входа `main.py` реализует консольный интерфейс: можно выбрать город (`--city moscow`), принудительно обновить данные, игнорируя кэш (`--refresh`), или отключить построение PNG-карт для ускорения (`--no-maps`).

3. Сбор больших пространственных данных из OSM

Для извлечения контуров зданий используется библиотека `osmnx`, модуль `features_from_place`, позволяющий по имени города получать `GeoDataFrame` с заданными OSM-тегами. В конфигурации теги заданы как `{"building": True}`, что соответствует выборке всех зданий в пределах административной границы города. Ключевая функция сбора данных:

```
import osmnx as ox
import geopandas as gpd
from config import OSM_BUILDING_TAGS, CRS_GEO

def fetch_buildings_osm(city_key: str, city_name: str) -> gpd.GeoDataFrame:
    gdf = ox.features_from_place(city_name, tags=OSM_BUILDING_TAGS)
    gdf = gdf[gdf.geometry.geom_type.isin(["Polygon",
    "MultiPolygon"])].copy()
    gdf = gdf.to_crs(CRS_GEO)
    return gdf
```

Листинг 1 — Загрузка зданий из OSM и фильтрация по типу геометрии

Далее данные фильтруются по полю `start_date`, из которого парсится год строительства здания. Это позволяет ограничить анализ заданным временным диапазоном, например 2010–2024 годы.

```
import pandas as pd
from config import YEAR_START, YEAR_END

def parse_year(value) -> int | None:
    if pd.isna(value):
        return None
    s = str(value).strip()
    try:
        return int(s[:4])
    except (ValueError, IndexError):
        return None

def filter_by_year(gdf, year_start=YEAR_START, year_end=YEAR_END):
    gdf = gdf.copy()
    gdf["year"] = gdf.get("start_date",
    pd.Series(dtype=object)).apply(parse_year)
    gdf = gdf[gdf["year"].notna()].copy()
    gdf["year"] = gdf["year"].astype(int)
    return gdf[(gdf["year"] >= year_start) & (gdf["year"] <=
    year_end)].copy()
```

Листинг 2 — Парсинг года строительства и фильтрация по диапазону лет

На этапе обогащения геометрии в метрической системе координат (например, UTM) вычисляется площадь каждого здания и координаты центроида.

```
def enrich_geometry(gdf, crs_metric="EPSG:32637"):
    gdf_m = gdf.to_crs(crs_metric)
    gdf["area_m2"] = gdf_m.geometry.area.round(1)
    gdf["centroid_lon"] = gdf.geometry.centroid.x
    gdf["centroid_lat"] = gdf.geometry.centroid.y
    return gdf
```

Листинг 3 — Обогащение геометрии: площадь и координаты центроида

Данные каждого города кэшируются в формате GeoPackage (*.gpkg), что позволяет при повторных запусках не обращаться к Overpass API.

4. Пространственно-временной анализ: KDE и аномалии

4.1. Метод Kernel Density Estimation

Ядерная оценка плотности (Kernel Density Estimation, KDE) используется для построения непрерывной поверхностной оценки интенсивности появления новых зданий по координатам их центроидов. Метод заключается в размещении сглаживающей функции (ядра) вокруг каждой точки и суммировании вкладов по регулярной сетке, что позволяет визуализировать «горячие точки» концентрации застройки. В модуле `analyzer.py` KDE реализована с помощью `scipy.stats.gaussian_kde`:

```
from scipy.stats import gaussian_kde
import numpy as np

def detect_point_anomalies(gdf, bandwidth=0.15, zscore_threshold=2.5):
    coords = np.vstack([
        gdf.geometry.centroid.x.values,
        gdf.geometry.centroid.y.values
    ])
    kde = gaussian_kde(coords, bw_method=bandwidth)
    density = kde(coords)

    mean = density.mean()
    std = density.std()
    z = (density - mean) / std if std > 0 else np.zeros_like(density)

    gdf = gdf.copy()
    gdf["kde_density"] = density.round(8)
    gdf["z_score"] = z.round(4)
    gdf["is_anomaly"] = z > zscore_threshold
    return gdf
```

Листинг 4 — Обнаружение аномалий методом KDE и z-score

Критерием аномальности служит порог по z-score: объекты с локальной плотностью выше среднего более чем на 2.5 стандартных отклонения считаются аномальными. Такой подход позволяет выделить районы с чрезмерно высокой концентрацией новых объектов по сравнению с фоном города.

4.2. Годовая статистика и кластеризация

Для оценки динамики строительной активности вычисляется статистика по годам: количество зданий, суммарная и средняя площадь, число аномальных объектов и относительные темпы роста.

```
def yearly_stats(gdf, year_start=YEAR_START, year_end=YEAR_END):
    rows = []
    for year in range(year_start, year_end + 1):
        yr = gdf[gdf["year"] == year]
        if len(yr) == 0:
            continue
        rows.append({
            "year": year,
            "count": len(yr),
            "total_area": yr["area_m2"].sum(),
            "mean_area": yr["area_m2"].mean().round(1),
            "anomalies": int(yr["is_anomaly"].sum()),
        })
    df = pd.DataFrame(rows)
    df["pct_change"] = df["count"].pct_change().mul(100).round(1)
    return df
```

Листинг 5 — Годовая статистика строительной активности

Для группировки аномалий в «горячие зоны» используется кластеризация методом DBSCAN по географическим координатам в радианах и метрике Haversine, что позволяет объединить близко расположенные аномальные здания в кластеры.

5. Визуализация и финальные результаты

5.1. Статические карты KDE и динамика

Модуль `visualizer.py` строит статические PNG-карты KDE для каждого года, используя `Matplotlib`. Для каждого среза создаётся карта, на которую накладывается цветной растр плотности и полупрозрачные точки зданий:

```
import matplotlib.pyplot as plt
from matplotlib.colors import Normalize

def plot_kde_year(gdf, year, city_key="city"):
    yr_gdf = gdf[gdf["year"] == year]
    coords = np.vstack([
        yr_gdf.geometry.centroid.x.values,
        yr_gdf.geometry.centroid.y.values
    ]).T
    xx, yy, density = compute_kde_grid(coords)
    fig, ax = plt.subplots(figsize=(10, 8), facecolor="#0a0a0f")
    ax.contourf(xx, yy, density, levels=20,
               cmap="plasma",
               norm=Normalize(vmin=0, vmax=density.max()),
               alpha=0.85)
    ax.scatter(coords[:, 0], coords[:, 1],
               s=1, c="#06b6d4", alpha=0.15, linewidths=0)
    plt.savefig(f"output/{city_key}_kde_{year}.png", dpi=120,
               bbox_inches="tight", facecolor="#0a0a0f")
```

Листинг 6 — Построение статической KDE-карты для одного года

Также строится график динамики: в верхней панели — количество новых зданий по годам, в нижней — число аномалий.

5.2. Интерактивная карта в Folium

Для визуализации аномальных зон в веб-формате применяется `Folium`, построенный поверх `Leaflet`, который хорошо подходит для интерактивных тепловых карт. На карту добавляется слой `heatmap` по координатам аномалий, а также кластер маркеров с подсказками по году, площади и `z-score`:

```
import folium
from folium.plugins import HeatMap, MarkerCluster

def make_interactive_map(gdf, city_key="city"):
    anom = gdf[gdf["is_anomaly"] == True]
    center_lat = gdf.geometry.centroid.y.mean()
    center_lon = gdf.geometry.centroid.x.mean()

    m = folium.Map(location=[center_lat, center_lon],
                   zoom_start=11,
```

```

tiles="CartoDB dark_matter")

heat_data = [
    [row.geometry.centroid.y,
     row.geometry.centroid.x,
     float(row["z_score"])]
    for _, row in anom.iterrows()
]
HeatMap(heat_data, radius=12, blur=8,
        min_opacity=0.4).add_to(m)

mc = MarkerCluster(name="Anomal'nye zdaniya").add_to(m)
for _, row in anom.head(50).iterrows():
    popup_text = (
        f"<b>God:</b> {row['year']}<br>"
        f"<b>Z-score:</b> {row['z_score']:.2f}<br>"
        f"<b>Ploshchad:</b> {row['area_m2']} m2"
    )
    folium.CircleMarker(
        location=[row.geometry.centroid.y,
                  row.geometry.centroid.x],
        radius=5,
        color="#06b6d4",
        fill=True,
        fill_color="#06b6d4",
        fill_opacity=0.7,
        popup=folium.Popup(popup_text, max_width=200)
    ).add_to(mc)

m.save(f"output/{city_key}_anomaly_map.html")

```

Листинг 7 — Построение графика динамики застройки и аномалий

5.3. CSV, Geopackage и текстовый отчет

Модуль reporter.py сохраняет:

1. CSV-файл с аномалиями (год, площадь, координаты, z-score, принадлежность к кластеру).
2. GeoPackage с обогащённым слоем зданий для последующего анализа в QGIS.
3. CSV со статистикой по годам.
4. Текстовый summary-отчёт, содержащий общие показатели, годы пиков активности и перечень топ-10 аномальных зон.

Фрагмент генерации текстового отчёта:

```

def generate_text_report(gdf, stats, city_key="city", map_paths=None):
    anom = gdf[gdf["is_anomaly"] == True]

```

```

peak_year = stats.loc[stats["count"].idxmax(), "year"]
peak_count = stats["count"].max()

report = f"""
1. ОБЩАЯ СТАТИСТИКА
Всего объектов:      {len(gdf):>10,}
Аномальных зон:      {len(anom):>10,}
Процент аномалий:    {100*len(anom)/max(len(gdf),1):>9.1f}%
Период анализа:      {stats['year'].min()} – {stats['year'].max()}

2. ПИКИ АКТИВНОСТИ
Максимум зданий за год: {peak_year} ({peak_count:},) об.)
"""

# + top-10 аномалий и список сгенерированных файлов

```

Листинг 8 — Интерактивная тепловая карта аномалий в Folium

6. Заключение

По итогам работы реализована сквозная система, которая:

1. автоматически собирает большие объёмы пространственных данных о зданиях из OSM для выбранных городов;
2. нормализует и обогащает геометрию зданиями, включая расчёт площадей и координат центроидов;
3. выполняет пространственно-временной анализ интенсивности застройки методом KDE и выделяет аномальные зоны по статистическому критерию z-score;
4. формирует набор выходных продуктов: статические карты плотности, интерактивную веб-карту, CSV/GeoPackage и текстовые аналитические отчёты.

С точки зрения строительной отрасли разработанная система может использоваться как прототип сервиса мониторинга городской застройки, способный помочь в выявлении районов с подозрительно высокой активностью (возможные незаконные стройки или спекулятивная застройка). С точки зрения программной реализации это законченный пример системы сбора и обработки больших пространственных данных, где демонстрируются практические навыки работы с OSM, GeoPandas, алгоритмами KDE и современными средствами визуализации.

Список использованных источников

1. Budde R. Kernel density analysis – a tool for the visualization of spatial distribution patterns [Электронный ресурс]. – URL: https://rural-urban.eu/files/sites/default/files/02_kernel%20density%20analysis_budde.pdf (дата обращения: 06.06.2026).
2. Heatmap на интерактивной карте с помощью folium // Хабр [Электронный ресурс]. – 2023. – 8 янв. – URL: <https://habr.com/ru/articles/709092/> (дата обращения: 06.06.2026).
3. OSMnx 2.1.0 documentation [Электронный ресурс]. – URL: <https://osmnx.readthedocs.io> (дата обращения: 06.06.2026).
4. Plotting Geographical Heatmaps using Python Folium Library // ByteScout Blog [Электронный ресурс]. – URL: <https://bytescout.com/blog/plotting-geographical-heatmaps-using-python-folium-library.html> (дата обращения: 06.06.2026).
5. Retrieving OpenStreetMap data // Python for GIS – Online Book [Электронный ресурс]. – URL: <https://pythongis.org/part2/chapter-09/nb/00-retrieving-osm-data.html> (дата обращения: 06.06.2026).
6. Visualizing Geospatial Data: Creating Heatmaps with Plotly and Folium in Python // CloudThat Blog [Электронный ресурс]. – 2024. – 24 Jun. – URL: <https://www.cloudthat.com/resources/blog/visualizing-geospatial-data-creating-heatmaps-with-plotly-and-folium-in-python> (дата обращения: 06.06.2026).
7. Understanding Kernel Density Estimation in Spatial Analysis // Spatial Eye Blog [Электронный ресурс]. – 2026. – 26 Apr. – URL: <https://spatial-eye.com/blog/spatial-analysis/understanding-kernel-density-estimation-in-spatial-analysis/> (дата обращения: 06.06.2026).